

Python — La visualisation de données

Document 10 — Matplotlib, la fondation

Série « Visualisation » · la bibliothèque fondatrice

Table des matières

- 1. Le modèle Figure / Axes
- 2. Première image : un tracé minimal
 - 2.1 L'approche pyplot (rapide, implicite)
 - 2.2 L'approche orientée objet (recommandée)
- 3. Un graphique complet
- 4. Les types de tracés courants
- 5. Plusieurs graphiques : subplots et GridSpec
 - 5.1 plt.subplots : une grille régulière
 - 5.2 GridSpec : des dispositions inégales
- 6. Styles et personnalisation
- 7. Annotations et texte combiné aux graphiques
- 8. Sauvegarde et export
- Récapitulatif
- Questions de vérification

1. Le modèle Figure / Axes

Tout, dans Matplotlib, repose sur deux objets qu'il faut distinguer dès le départ. La Figure est la page : le conteneur global. Les Axes sont une zone de tracé à l'intérieur de cette page : un graphique, avec son axe x, son axe y, son titre. Une Figure peut contenir plusieurs Axes. Bien séparer les deux évite la plupart des confusions.

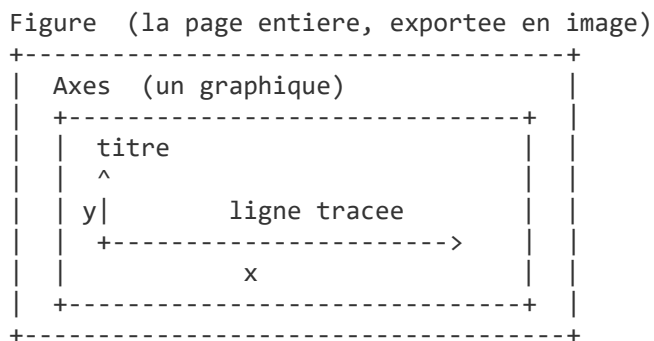


Figure. Le conteneur de plus haut niveau. Il porte la taille (figsize), la résolution (dpi), le fond, et un éventuel titre global (suptitle). C'est la Figure qu'on exporte vers un fichier image.

Axes. La zone où l'on trace réellement. Elle porte ses deux axes, son titre, sa légende et sa grille. La plupart des méthodes de tracé (plot, scatter, bar...) sont des méthodes d'un objet Axes.

Objet	Rôle
Figure	La page entière : conteneur de tous les graphiques. C'est elle qu'on sauvegarde.
Axes	Une zone de tracé (un graphique). Une Figure peut en contenir plusieurs.

Objet	Rôle
Axis	UN axe — x ou y : ses graduations, son échelle, son libellé.
Artist	Tout élément dessinnable : une ligne, un texte, un rectangle.

Pourquoi ça compte. Ne pas confondre Axes (la zone de tracé, un graphique — le pluriel est trompeur) avec Axis (un seul axe, x ou y). C'est la confusion n°1 des débutants.

2. Première image : un tracé minimal

Deux façons d'utiliser Matplotlib coexistent. Il faut connaître les deux — on les rencontre partout — mais en préférer une.

2.1 L'approche pyplot (rapide, implicite)

pyplot maintient une figure « courante » en arrière-plan et lui applique chaque commande. C'est pratique pour un test éclair, dans un notebook par exemple.

```
import matplotlib.pyplot as plt

prix = [100, 102, 101, 105, 107]
plt.plot(prix)
plt.title("Prix de cloture")
plt.show()
```

Court, mais l'état global devient vite ingérable : dès qu'on a plusieurs graphiques, on ne sait plus auquel chaque commande s'applique.

2.2 L'approche orientée objet (recommandée)

On crée explicitement la Figure et les Axes avec `plt.subplots()`, puis on appelle les méthodes sur l'objet `ax`. On sait toujours sur quel graphique on agit. C'est la convention professionnelle, suivie dans toute la série.

```
import matplotlib.pyplot as plt

prix = [100, 102, 101, 105, 107]
fig, ax = plt.subplots()      # une figure, un axes
ax.plot(prix)
ax.set_title("Prix de cloture")
fig.savefig("prix.png")
```

`fig, ax = plt.subplots()` est le point de départ de presque tout code Matplotlib sérieux : une Figure et l'Axes qu'elle contient, récupérés d'un coup.

Pourquoi ça compte. Privilégier l'approche orientée objet (`fig, ax`). Le code reste explicite et lisible dès qu'il y a plusieurs graphiques — et c'est la convention de toute la série.

3. Un graphique complet

Un tracé brut se lit mal. Un graphique livrable porte un titre, des axes nommés, une légende et une grille discrète. Tout cela se règle par les méthodes `set_*` de l'objet Axes.

```

import matplotlib.pyplot as plt

jours = [1, 2, 3, 4, 5]
es = [100, 102, 101, 105, 107]
nq = [98, 99, 103, 104, 106]

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(jours, es, label="ES", color="steelblue", linewidth=2)
ax.plot(jours, nq, label="NQ", color="orange", linestyle="--")

ax.set_title("Comparaison ES / NQ")
ax.set_xlabel("Jour")
ax.set_ylabel("Prix de cloture")
ax.legend() # a partir des label=
ax.grid(True, alpha=0.3) # grille discrete

fig.tight_layout() # ajuste les marges
fig.savefig("comparaison.png", dpi=150)

```

Méthode	Rôle
<code>ax.plot(x, y)</code>	Tracer une ligne (série de prix dans le temps).
<code>ax.set_title(...)</code>	Titre du graphique.
<code>ax.set_xlabel / set_ylabel</code>	Libellé de l'axe x / de l'axe y.
<code>ax.legend()</code>	Légende, construite à partir des label=.
<code>ax.grid(True)</code>	Affiche la grille (alpha pour la discrétion).
<code>ax.set_xlim / set_ylim</code>	Bornes affichées sur chaque axe.

Le couple label= / legend(). La légende se construit à partir des label= passés à chaque tracé. Sans label, `ax.legend()` n'a rien à afficher : c'est l'oubli le plus fréquent.

4. Les types de tracés courants

Chaque type de graphique a sa méthode sur l'objet Axes. Le choix dépend de la donnée et de la question posée.

Méthode	Type	Quand l'utiliser
<code>ax.plot</code>	Ligne	Évolution dans le temps (série de prix).
<code>ax.scatter</code>	Nuage de points	Relation entre deux variables (corrélation).
<code>ax.bar / barh</code>	Barres	Comparer des catégories (volumes par actif).
<code>ax.hist</code>	Histogramme	Distribution d'une variable (rendements).
<code>ax.fill_between</code>	Aire	Zone : bandes, intervalle de confiance.
<code>ax.boxplot</code>	Boîte à moustaches	Dispersion et valeurs aberrantes.

```

# relation entre deux series de rendements
fig, ax = plt.subplots()
ax.scatter(rendements_es, rendements_nq, alpha=0.5)

```

```

ax.set_xlabel("Rendement ES")
ax.set_ylabel("Rendement NQ")

# distribution des rendements ES
fig2, ax2 = plt.subplots()
ax2.hist(rendements_es, bins=30, color="steelblue")

```

Pourquoi ça compte. La méthode change, mais la structure reste identique : `fig, ax = plt.subplots()` puis `ax.<type>(...)`. Apprendre un type de tracé, c'est presque les apprendre tous.

5. Plusieurs graphiques : subplots et GridSpec

5.1 plt.subplots : une grille régulière

Pour disposer plusieurs graphiques en grille régulière, on passe le nombre de lignes et de colonnes. La variable retournée devient alors un tableau d'Axes.

```

fig, axes = plt.subplots(2, 2, figsize=(10, 6))

axes[0, 0].plot(prix_es)
axes[0, 1].plot(prix_nq)
axes[1, 0].hist(rendements_es)
axes[1, 1].scatter(rendements_es, rendements_nq)

```

```
fig.tight_layout()
```

`axes` est ici un tableau 2x2 ; on accède à chaque case par `axes[ligne, colonne]`. `tight_layout()` évite que les graphiques ne se chevauchent.

5.2 GridSpec : des dispositions inégales

Quand les graphiques n'ont pas la même taille — un grand graphique de prix au-dessus, un petit volume en dessous — `GridSpec` donne le contrôle fin des proportions.

```

import matplotlib.gridspec as gridspec

fig = plt.figure(figsize=(10, 6))
gs = gridspec.GridSpec(2, 1, height_ratios=[3, 1])

ax_prix = fig.add_subplot(gs[0])    # 3/4 de la hauteur
ax_vol  = fig.add_subplot(gs[1])    # 1/4 de la hauteur

ax_prix.plot(prix)
ax_vol.bar(jours, volumes)

```

`height_ratios=[3, 1]` donne au graphique de prix trois fois la hauteur du volume — exactement la disposition d'un graphique boursier classique.

Pourquoi ça compte. `subplots()` pour une grille régulière ; `GridSpec` (avec `height_ratios` / `width_ratios`) pour des proportions inégales. Le duo `prix + volume` en est l'usage emblématique.

6. Styles et personnalisation

Matplotlib se personnalise à trois niveaux, du plus local au plus global.

Par appel. color, linewidth, linestyle, marker, alpha passés directement à plot/scatter. N'affecte que ce tracé-là.

Feuille de style. plt.style.use("ggplot") applique un thème complet (couleurs, grille, polices) en une ligne, pour toute la suite.

rcParams. plt.rcParams modifie les valeurs par défaut globales (taille de police, dpi...) pour toute la session.

```
import matplotlib.pyplot as plt

plt.style.use("ggplot")          # un theme complet
plt.rcParams["figure.dpi"] = 120 # default global
plt.rcParams["font.size"] = 11

fig, ax = plt.subplots()
ax.plot(prix, color="#1A4A6B", linewidth=2, marker="o")
```

Paramètre	Effet	Exemples
color	Couleur du tracé	"steelblue", "#1A4A6B"
linewidth	Épaisseur de ligne	1, 2, 0.5
linestyle	Style de ligne	"-", "--", ":",
marker	Marqueur de point	"o", "s", "^"
alpha	Transparence	0.3 (0 = invisible, 1 = opaque)

Pourquoi ça compte. Pour lister les thèmes disponibles : `plt.style.available`. « ggplot » et les styles « seaborn » donnent un rendu soigné sans effort. Le document 999 (cheat sheets) recense colormap, marqueurs et rcParams.

7. Annotations et texte combiné aux graphiques

On enrichit un graphique avec du texte : une annotation pointant un événement, un seuil de référence, un commentaire. Trois outils suffisent dans la plupart des cas.

Méthode	Rôle
<code>ax.text(x, y, s)</code>	Placer du texte à une position (coordonnées des données).
<code>ax.annotate(s, xy, xytext)</code>	Texte + flèche pointant un point précis.
<code>ax.axhline / ax.vline</code>	Ligne horizontale / verticale de repère (seuil, date).

```
fig, ax = plt.subplots()
ax.plot(jours, prix)

# annoter le sommet avec une fleche
ax.annotate("Plus haut",
            xy=(4, 107), xytext=(2, 110),
            arrowprops=dict(arrowstyle="->"))

# seuil de support
ax.axhline(100, color="red", linestyle="--", alpha=0.5)
```

```
ax.text(1, 100.5, "Support", color="red")
```

xy est le point visé (la pointe de la flèche), xytext la position du texte.

Pourquoi ça compte. Par défaut, x et y sont les coordonnées des données (un prix, un jour). Pour positionner en proportion de l'axe (0 à 1, indépendamment des données), passer `transform=ax.transAxes`.

8. Sauvegarde et export

Le livrable final est un fichier. `fig.savefig` écrit la figure ; le format est déduit de l'extension du nom de fichier.

```
fig.savefig("rapport.png", dpi=300)          # raster haute resolution
fig.savefig("rapport.pdf")                   # vectoriel, pour impression
fig.savefig("rapport.svg")                   # vectoriel, pour le web
fig.savefig("rapport.png", bbox_inches="tight",
            transparent=True)                 # rogne les marges, fond transparent
```

Format	Nature	Usage
PNG	Raster (pixels)	Rapport, présentation, web ; régler le dpi.
PDF	Vectoriel	Impression, document : net à toute échelle.
SVG	Vectoriel	Web, édition ultérieure (Inkscape, Illustrator).
JPG	Raster compressé	Photos ; à éviter pour les graphiques (artefacts).

dpi. Résolution en points par pouce : 150 pour l'écran, 300 pour l'impression. Sans effet sur les formats vectoriels (PDF, SVG), nets à toute taille.

bbox_inches="tight". Rogne les marges blanches superflues autour de la figure — utile avant d'insérer l'image dans un rapport.

À retenir : appeler `savefig` avant `plt.show()`. `show()` peut vider la figure ; sauvegarder d'abord garantit un fichier complet. Dans une boucle, fermer chaque figure avec `plt.close(fig)` pour libérer la mémoire.

Pourquoi ça compte. C'est l'étape 4 du pipeline (la sortie) qui décide du format : PDF vectoriel pour un rapport imprimé, PNG en `dpi=150` pour l'écran ou une présentation.

Récapitulatif

- Figure / Axes.** La Figure est la page, les Axes une zone de tracé. `fig, ax = plt.subplots()` est le point de départ.
- Deux approches.** `pyplot` (implicite, rapide) et orientée objet (explicite, recommandée). On préfère `fig, ax`.
- Graphique complet.** Titre, libellés d'axes, légende (via `label=`), grille, `tight_layout`.
- Types de tracés.** `plot`, `scatter`, `bar`, `hist`, `fill_between...` même structure, méthode différente.
- Plusieurs graphiques.** `subplots(n, m)` pour une grille régulière ; `GridSpec` pour des proportions inégales (prix + volume).
- Styles.** Par appel, par feuille de style (`plt.style.use`) ou par `rcParams` globaux.
- Texte.** `text`, `annotate` (avec flèche), `axhline` / `axvline` pour les repères.

8. Export. `fig.savefig` ; PDF vectoriel pour l'impression, PNG (dpi) pour l'écran. Sauvegarder avant `show()`.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Q1. Quelle est la différence entre une Figure et des Axes ?

Réponse. La Figure est le conteneur global (la page entière), qu'on exporte en image. Les Axes sont une zone de tracé à l'intérieur : un graphique avec ses axes x/y, son titre, sa légende. Une Figure peut contenir plusieurs Axes.

Q2. Pourquoi préférer l'approche orientée objet (`fig, ax`) à `pyplot` ?

Réponse. Parce qu'elle est explicite : on sait toujours sur quel graphique on agit. Dès qu'il y a plusieurs Axes, l'état global implicite de `pyplot` devient ingérable. `fig, ax = plt.subplots()` est la convention professionnelle.

Q3. Comment afficher une légende, et qu'est-ce qui la remplit ?

Réponse. `ax.legend()`. Elle se construit à partir des `label=` passés à chaque tracé (`ax.plot(..., label="ES")`). Sans label, la légende n'a rien à afficher.

Q4. Quand utiliser `GridSpec` plutôt que `plt.subplots` ?

Réponse. Quand les graphiques doivent avoir des tailles inégales. `subplots(n, m)` fait une grille régulière ; `GridSpec` (avec `height_ratios`) permet un grand graphique de prix au-dessus d'un petit volume — la disposition boursière classique.

Q5. Quels sont les trois niveaux de personnalisation d'un graphique ?

Réponse. Par appel (`color`, `linewidth...` sur un tracé précis), par feuille de style globale (`plt.style.use("ggplot")`), et par `rcParams` (`plt.rcParams`, défauts de toute la session).

Q6. Quel format d'export pour un rapport imprimé, et pourquoi ?

Réponse. PDF (ou SVG) : ce sont des formats vectoriels, nets à toute échelle, idéaux pour l'impression — le dpi n'a aucun effet sur eux. Pour l'écran ou le web, on exporte un PNG en réglant le dpi (150 écran, 300 impression).

Document suivant (20) : *Pandas (.plot) — tracer directement depuis un DataFrame, en une ligne, pour explorer vite avant de peaufiner avec Matplotlib.*